AWT Assignments-3 Code

Krishna Patil
PRN: 125M1H025
Subject: AWT

Task 1

Question 1

Problem Statement

Templating with JSX...

Code

Filename: Assignment1.jsx

```
import { useState } from "react";

function Assignment1() {
  const items = ["React", "JavaScript", "JSX", "Vite", "CSS"];

  const currentHour = new Date().getHours();

  const [showList, setShowList] = useState(true);

  return (
    <div>
      <div className="assignment-heading">
        <h2>Assignment 1: Templating with JSX</h2>
        <p>Working with JSX, expressions and dynamic rendering.</p>
      </div>

      {/* Simple JSX Component */}
      <section className="card">
        <h3>1. JSX Component</h3>

        <div className="jsx-box">
          <h1>Welcome to React</h1>
          <p>
            React is a JavaScript library used to build interactive user
            interfaces.
          </p>
        </div>
      </section>

      {/* Dynamic List */}
      <section className="card">
        <h3>2. Dynamic List using map()</h3>

        <button
          className="secondary-button"
          onClick={() => setShowList(!showList)}
        >
          {showList ? "Hide List" : "Show List"}
        </button>

        {showList && (
          <ul className="item-list">
            {items.map((item, index) => (
              <li key={index}>
                {index + 1}. {item}
              </li>
            ))}
          </ul>
        )}
      </section>

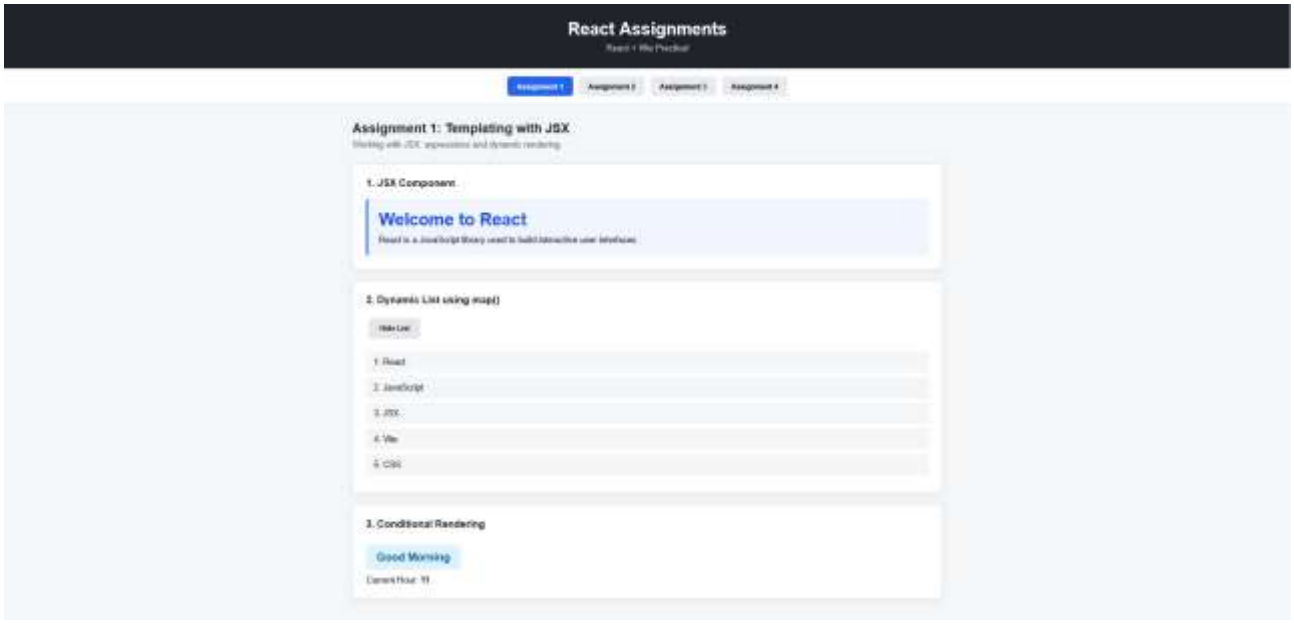
      {/* Conditional Rendering */}
      <section className="card">
        <h3>3. Conditional Rendering</h3>

        <div className="greeting">
          {currentHour < 12 ? "Good Morning" : "Good Evening"}
        </div>

        <p>
          Current Hour: <strong>{currentHour}</strong>
        </p>
      </section>
```

```
    </div>
  );
}

export default Assignment1;
```



Task 2

Question 1

Problem Statement

State and Lifecycle…

Code

Filename: Assignment2.jsx

```
import React from "react";

/* Counter using componentDidMount() */
class Counter extends React.Component {
  constructor(props) {
    super(props);

    this.state = {
      count: 0,
    };
  }

  componentDidMount() {
    this.timer = setInterval(() => {
      this.setState({
        count: this.state.count + 1,
      });
    }, 1000);
  }

  componentWillUnmount() {
    clearInterval(this.timer);
  }

  render() {
    return (
      <div className="demo-box">
        <h3>1. Counter using componentDidMount()</h3>

        <div className="counter-value">
          {this.state.count}
        </div>

        <p>Counter increases automatically every second.</p>
      </div>
    );
  }
}

/* Clock Component */
class Clock extends React.Component {
  constructor(props) {
    super(props);

    this.state = {
      time: new Date(),
    };
  }

  componentDidMount() {
    this.timer = setInterval(() => {
      this.setState({
        time: new Date(),
      });
    }, 1000);
  }

  componentWillUnmount() {
    clearInterval(this.timer);
  }

  render() {
    return (
      <div className="demo-box">
        <h3>2. Digital Clock</h3>

        <div className="clock">
          {this.state.time.toLocaleTimeString()}
        </div>

        <p>The clock updates every second.</p>
      </div>
    );
  }
}

/* Button Click Counter */
```

```
class ClickCounter extends React.Component {
  constructor(props) {
    super(props);

    this.state = {
      clicks: 0,
    };
  }

  handleClick = () => {
    this.setState({
      clicks: this.state.clicks + 1,
    });
  };

  render() {
    return (
      <div className="demo-box">
        <h3>3. Button Click Counter</h3>

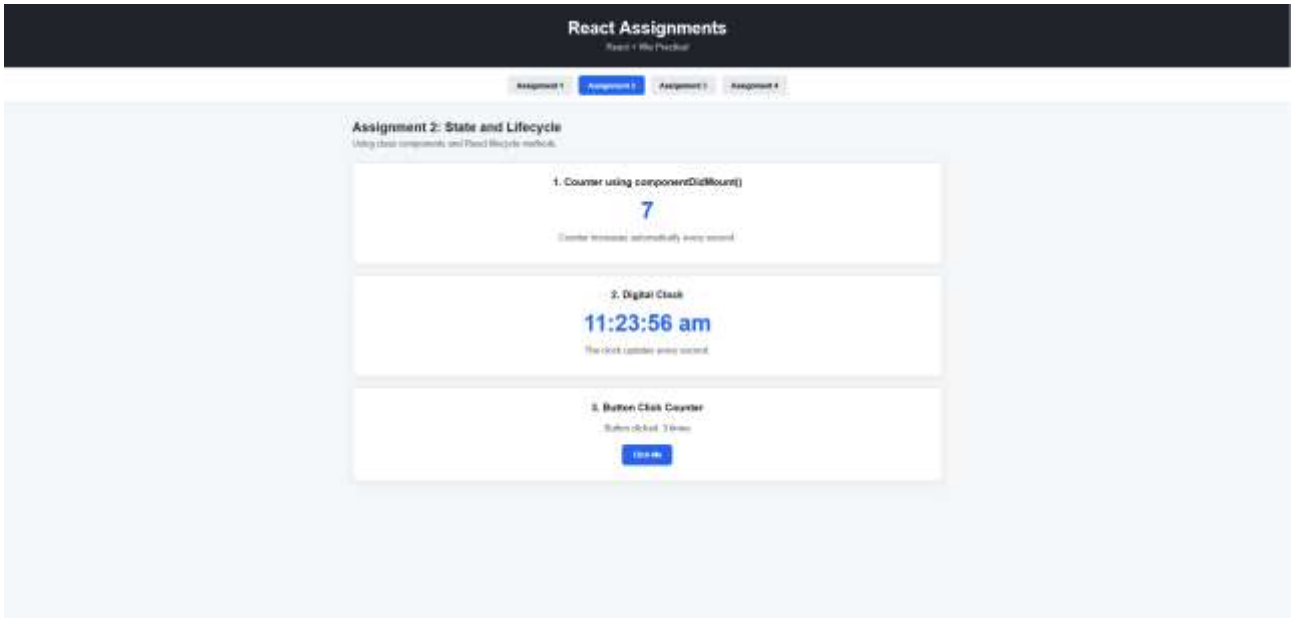
        <p>
          Button clicked:
          <strong> {this.state.clicks}</strong> times
        </p>

        <button className="primary-button" onClick={this.handleClick}>
          Click Me
        </button>
      </div>
    );
  }
}

function Assignment2() {
  return (
    <div>
      <div className="assignment-heading">
        <h2>Assignment 2: State and Lifecycle</h2>
        <p>Using class components and React lifecycle methods.</p>
      </div>

      <Counter />
      <Clock />
      <ClickCounter />
    </div>
  );
}

export default Assignment2;
```



Task 3

Question 1

Problem Statement

React Hooks...

Code

Filename: Assignment3.jsx

```
import { useEffect, useState } from "react";

function Assignment3() {
  const [entered, setEntered] = useState(0);
  const [exited, setExited] = useState(0);

  const currentVisitors = entered - exited;

  // useEffect runs whenever the visitor count changes
  useEffect(() => {
    console.log(
      `Visitor count changed. Current visitors: ${currentVisitors}`
    );

    document.title = `Visitors: ${currentVisitors}`;
  }, [currentVisitors]);

  const handleEnter = () => {
    setEntered((prev) => prev + 1);
  };

  const handleExit = () => {
    if (currentVisitors > 0) {
      setExited((prev) => prev + 1);
    }
  };

  const handleReset = () => {
    setEntered(0);
    setExited(0);
  };

  // Message is derived directly from currentVisitors
  const message =
    currentVisitors === 0
      ? "There are currently no visitors inside."
      : `Visitor count changed. There are ${currentVisitors} visitor${
          currentVisitors !== 1 ? "s" : ""
        } currently inside.`;

  return (
    <div>
      <div className="assignment-heading">
        <h2>Assignment 3: Visitor Counter Dashboard</h2>
        <p>Using useState() and useEffect().</p>
      </div>

      <div className="visitor-dashboard">
        <div className="visitor-card">
          <h3>Visitors Entered</h3>
          <div className="visitor-number">{entered}</div>
        </div>

        <div className="visitor-card">
          <h3>Visitors Exited</h3>
          <div className="visitor-number">{exited}</div>
        </div>

        <div className="visitor-card current">
          <h3>Current Visitors</h3>
          <div className="visitor-number">{currentVisitors}</div>
        </div>
      </div>

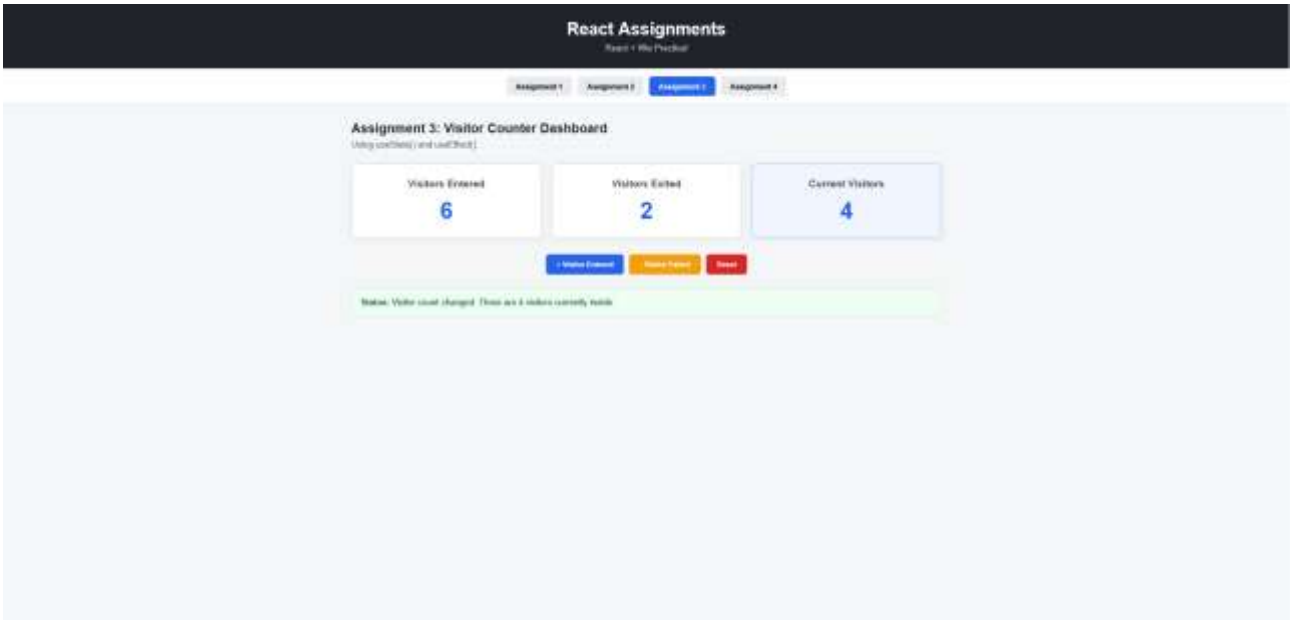
      <div className="visitor-controls">
        <button className="primary-button" onClick={handleEnter}>
          + Visitor Entered
        </button>

        <button className="warning-button" onClick={handleExit}>
          - Visitor Exited
        </button>

        <button className="danger-button" onClick={handleReset}>
          Reset
        </button>
      </div>
```

```
      <div className="message-box">
        <strong>Status:</strong> {message}
      </div>
    </div>
  );
}

export default Assignment3;
```



Task 4

Question 1

Problem Statement

React Forms...

Code

Filename: Assignment4.jsx

```
import { useState } from "react";

function Assignment4() {
  const initialForm = {
    name: "",
    dob: "",
    gender: "",
    email: "",
    mobile: "",
    address: "",
    city: "",
    state: "",
    course: "",
    qualification: "",
    percentage: "",
  };

  const [formData, setFormData] = useState(initialForm);
  const [errors, setErrors] = useState({});
  const [submittedData, setSubmittedData] = useState(null);

  const handleChange = (event) => {
    const { name, value } = event.target;

    setFormData({
      ...formData,
      [name]: value,
    });

    // Remove error when user starts correcting the field
    setErrors({
      ...errors,
      [name]: "",
    });
  };

  const validate = () => {
    const newErrors = {};
```

```
  // Name validation
  if (!formData.name.trim()) {
    newErrors.name = "Name cannot be empty.";
  }

  // Email validation
  const emailPattern = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;

  if (!formData.email.trim()) {
    newErrors.email = "Email is required.";
  } else if (!emailPattern.test(formData.email)) {
    newErrors.email = "Please enter a valid email address.";
  }

  // Mobile validation
  const mobilePattern = /^[0-9]{10}$/;

  if (!formData.mobile.trim()) {
    newErrors.mobile = "Mobile number is required.";
  } else if (!mobilePattern.test(formData.mobile)) {
    newErrors.mobile = "Mobile number must contain exactly 10 digits.";
  }

  // Percentage validation
  if (formData.percentage === "") {
    newErrors.percentage = "Percentage is required.";
  } else if (
    Number(formData.percentage) < 0 ||
    Number(formData.percentage) > 100
  ) {
    newErrors.percentage = "Percentage must be between 0 and 100.";
  }

  setErrors(newErrors);

  return Object.keys(newErrors).length === 0;
};

const handleSubmit = (event) => {
  event.preventDefault();

  if (validate()) {
    setSubmittedData(formData);
  }
};

const handleReset = () => {
  setFormData(initialForm);
  setErrors({});
  setSubmittedData(null);
};

return (
  <div>
    <div className="assignment-heading">
      <h2>Assignment 4: Student Admission Form</h2>
      <p>Controlled React form with validation.</p>
    </div>

    <form className="admission-form" onSubmit={handleSubmit}>

      {/* Personal Information */}
      <div className="form-section">
        <h3>Personal Information</h3>

        <div className="form-group">
          <label>Student Name</label>

          <input
            type="text"
            name="name"
            value={formData.name}
            onChange={handleChange}
            placeholder="Enter student name"
          />

          {errors.name && (
            <span className="error">{errors.name}</span>
          )}
        </div>

        <div className="form-row">

          <div className="form-group">
            <label>Date of Birth</label>

            <input
              type="date"
              name="dob"
              value={formData.dob}
              onChange={handleChange}
            />
          </div>

          <div className="form-group">
            <label>Gender</label>
```

```
      <div className="radio-group">
        <label>
          <input
            type="radio"
            name="gender"
            value="Male"
            checked={formData.gender === "Male"}
            onChange={handleChange}
          />
          Male
        </label>

        <label>
          <input
            type="radio"
            name="gender"
            value="Female"
            checked={formData.gender === "Female"}
            onChange={handleChange}
          />
          Female
        </label>

        <label>
          <input
            type="radio"
            name="gender"
            value="Other"
            checked={formData.gender === "Other"}
            onChange={handleChange}
          />
          Other
        </label>
      </div>
    </div>

  </div>
</div>

{/* Contact Information */}
<div className="form-section">
  <h3>Contact Information</h3>

  <div className="form-row">

    <div className="form-group">
      <label>Email</label>

      <input
        type="email"
        name="email"
        value={formData.email}
        onChange={handleChange}
        placeholder="student@example.com"
      />

      {errors.email && (
        <span className="error">{errors.email}</span>
      )}
    </div>

    <div className="form-group">
      <label>Mobile Number</label>

      <input
        type="tel"
        name="mobile"
        value={formData.mobile}
        onChange={handleChange}
        placeholder="10 digit mobile number"
      />

      {errors.mobile && (
        <span className="error">{errors.mobile}</span>
      )}
    </div>

  </div>

  <div className="form-group">
    <label>Address</label>

    <textarea
      name="address"
      value={formData.address}
      onChange={handleChange}
      placeholder="Enter complete address"
      rows="4"
    />
  </div>

  <div className="form-row">

    <div className="form-group">
      <label>City</label>

      <input
        type="text"
```

```
          name="city"
          value={formData.city}
          onChange={handleChange}
        />
      </div>

      <div className="form-group">
        <label>State</label>

        <select
          name="state"
          value={formData.state}
          onChange={handleChange}
        >
          <option value="">Select State</option>
          <option value="Maharashtra">Maharashtra</option>
          <option value="Gujarat">Gujarat</option>
          <option value="Karnataka">Karnataka</option>
          <option value="Goa">Goa</option>
          <option value="Delhi">Delhi</option>
          <option value="Other">Other</option>
        </select>
      </div>

    </div>
  </div>

  {/* Academic Information */}
  <div className="form-section">
    <h3>Academic Information</h3>

    <div className="form-row">

      <div className="form-group">
        <label>Course</label>

        <select
          name="course"
          value={formData.course}
          onChange={handleChange}
        >
          <option value="">Select Course</option>
          <option value="B.Tech">B.Tech</option>
          <option value="BCA">BCA</option>
          <option value="B.Sc">B.Sc</option>
          <option value="MCA">MCA</option>
          <option value="M.Tech">M.Tech</option>
        </select>
      </div>

      <div className="form-group">
        <label>Qualification</label>

        <select
          name="qualification"
          value={formData.qualification}
          onChange={handleChange}
        >
          <option value="">Select Qualification</option>
          <option value="12th">12th</option>
          <option value="Diploma">Diploma</option>
          <option value="Bachelor's Degree">
            Bachelor's Degree
          </option>
          <option value="Master's Degree">
            Master's Degree
          </option>
        </select>
      </div>

    </div>

    <div className="form-group">
      <label>Percentage</label>

      <input
        type="number"
        name="percentage"
        value={formData.percentage}
        onChange={handleChange}
        min="0"
        max="100"
        step="0.01"
        placeholder="Enter percentage"
      />

      {errors.percentage && (
        <span className="error">{errors.percentage}</span>
      )}
    </div>
  </div>

  {/* Buttons */}
  <div className="form-buttons">
    <button type="submit" className="primary-button">
      Submit Application
    </button>
```

```
            <button
              type="button"
              className="secondary-button"
              onClick={handleReset}
            >
              Reset
            </button>
          </div>

        </form>

        {/* Submitted Information */}
        {submittedData && (
          <div className="submitted-data">

            <h3>Application Submitted Successfully</h3>

            <div className="result-grid">
              <p>
                <strong>Name:</strong> {submittedData.name}
              </p>

              <p>
                <strong>Date of Birth:</strong> {submittedData.dob || "N/A"}
              </p>

              <p>
                <strong>Gender:</strong> {submittedData.gender || "N/A"}
              </p>

              <p>
                <strong>Email:</strong> {submittedData.email}
              </p>

              <p>
                <strong>Mobile:</strong> {submittedData.mobile}
              </p>

              <p>
                <strong>Address:</strong> {submittedData.address || "N/A"}
              </p>

              <p>
                <strong>City:</strong> {submittedData.city || "N/A"}
              </p>

              <p>
                <strong>State:</strong> {submittedData.state || "N/A"}
              </p>

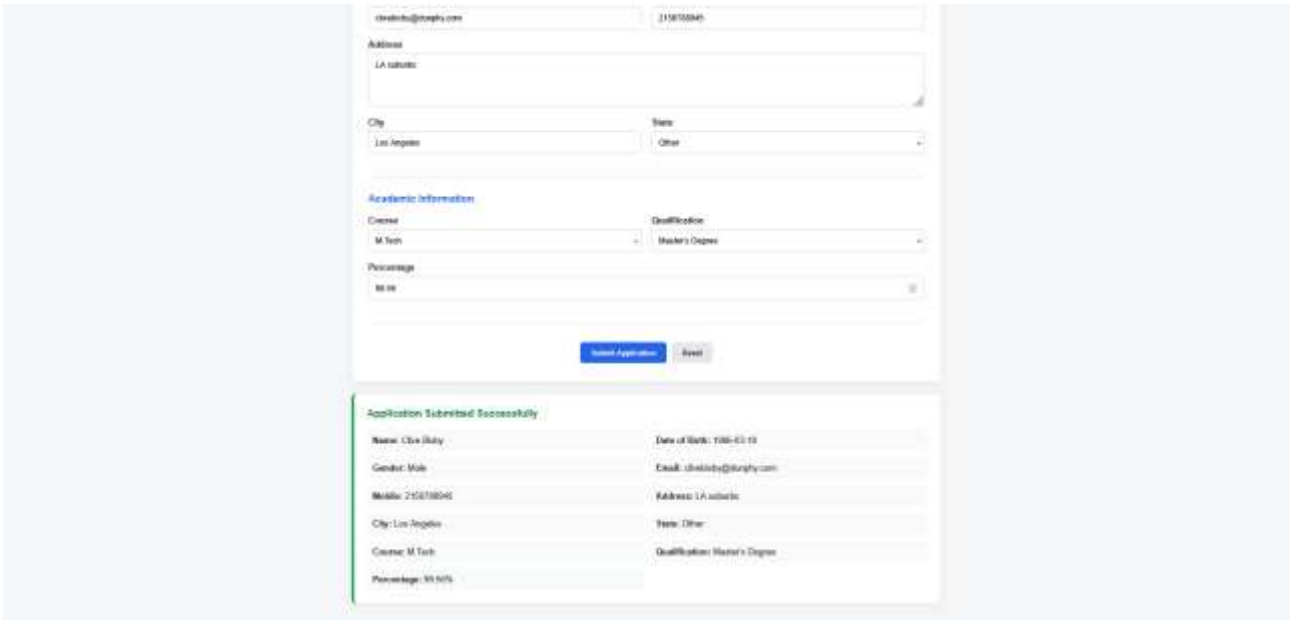
              <p>
                <strong>Course:</strong> {submittedData.course || "N/A"}
              </p>

              <p>
                <strong>Qualification:</strong>{" "}
                {submittedData.qualification || "N/A"}
              </p>

              <p>
                <strong>Percentage:</strong> {submittedData.percentage}%
              </p>
            </div>

          </div>
        )}
      </div>
    );
}

export default Assignment4;
```


Address
LA suburbs
City
Los Angeles
State
Other
Academic Information
Course
M.Tech
Qualification
Master's Degree
Percentage
99.99
Application Submitted Successfully
Name: Clive Ruby
Gender: Male
Mobile: 2156700045
City: Los Angeles
Course: M.Tech
Percentage: 99.99%